



Assessment Report INM702

Group N°: 1

Author: Bruno Santomé Antolín

Github: BrunoSantome

Github repository:

https://github.com/BrunoSantome/Programming_math_Ai_Assessment_Msc_Ai

Index

Index	2
1. Introduction	3
1.1 Datasets	3
Task 1: Sloan Digital Sky Survey - DR18	3
Task 2: Skyview Multi-Landscape Aerial Imagery Dataset	3
2. Task 1 - Building a Neural Network from scratch.	3
2.1 Activation functions	3
Relu / Sigmoid	4
Softmax	4
2.2 Fully parametrizable neural network	4
2.2.1 Architecture of the network	5
2.2.2 Basic hyperparameters	5
2.2.3 Regularisation	5
1. Dropout / Inverted Dropout	5
2. L1/L2 Regularisation	5
2.2.4 Optimizers	6
1. Momentum	6
2. Mini Batch Gradient Descent	6
2.3 Results and Discussion	6
Different Architectures	6
Optimizers	7
Relu vs Sigmoid	7
Regularisation techniques	8
Final Results	8
3. Task 2 - Building Neural networks with PyTorch	9
3.1 First Model: Neural Network	9
3.2 Architecture Improvement: Convolutional Neural Network	9
3.3 Results and discussion	9
Regular Neural Network Results	9
Convolutional Neural Networks Results	10
Comparison of final results between the two models:	11
4. References	12
References for Task 1	12
References for Task 2	13

1. Introduction

In this assessment report we present the dataset selection, the procedure and the results carried out for the two tasks that compose it. I will first present the choice and motivations for the two datasets used in the tasks, then describe the first task, and finally the second task.

1.1 Datasets

Task 1: **Sloan Digital Sky Survey - DR18**

The dataset is composed of 100.000 observations from the Data Release (DR) 18 of the Sloan Digital Sky Survey (SDSS), which is one of the most comprehensive astronomical surveys ever conducted, DR18 is the 18th and final public data release from SDSS-IV. There are three kinds of observations; Galaxies, Stars or Quasars (Quasi-Stellar Object - QSO). The celestial objects are described by 42 features which represent all kind of useful information for their study, such as, their sizes, morphologies, how they evolve over time, photometric properties, brightness, ... For a detailed explanation of the features visit the source of the dataset [here](#). The main problem with the dataset is the imbalance of the classes. 52.34% of the celestial objects are galaxies, 37.23% are stars and the remaining 10.43% are quasars. This imbalance makes the model start learning with a very high accuracy. The main motivation of the choice is my high interest in the fields of astronomy and astrophysics. My main objective is to combine A.I. techniques with these fields. I believed this was a great opportunity to gain some real insight into the domain since DR18 is the most complete, accurate, and up-to-date SDSS dataset. Is the description of the objects enough to make a model learn to classify correctly? What are the objects which the model might struggle more to detect?

Task 2: **Skyview Multi-Landscape Aerial Imagery Dataset**

The dataset is composed of 12.000 images of aerial landscapes, there are 15 different landscapes in total and 800 images at a resolution of 256x256 pixels in each category, making it a very good balanced dataset to work in. The categories are; Agriculture, Airport, Beach, City, Desert, Forest, Grassland, Highway, Lake, Mountain, Parking, Port, Railway, Residential and River. This dataset is a fusion of images sourced from the publicly available AID and NWPU-Resisc45 datasets. I found the dataset well suited for this task because it did not seem to need a lot of pre-processing, it was highly balanced and organized. Besides, I was attracted by the beauty of combining aerial landscapes images with artificial intelligence, merging satellite images with complex algorithms. Is there any landscape in particular that is harder for a neuronal network to detect? How could I create a neuronal network that could correctly classify the different landscapes?

2. Task 1 - Building a Neural Network from scratch.

2.1 Activation functions

I implemented these functions based on their respectful formulas and the implementation viewed during the lab of week 7 of the course.

Relu / Sigmoid

Sigmoid function: $y = \frac{1}{1 + e^{-x}}$. Relu function: $f(x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases}$

The Sigmoid function maps any real-valued number into the range (0, 1). The ReLU function only takes the positive numbers, the negative ones it changes them to 0. Both functions are useful to prevent linearity in the network however there are advantages and disadvantages in both cases:

Generally the Relu function is used more than the Sigmoid function for the hidden layers of the network because of two main reasons: firstly there is a less likelihood of the gradients to vanish because when a positive value is fed (x equal or greater than 0) the gradient becomes a constant value (input=output). In contrast the gradient of the sigmoid becomes increasingly small as the absolute value of x increases (which causes the gradient to vanish). Relu is faster and produces faster learning for this reason. At the same time the sigmoid function has to compute an expensive exponential operation whilst the Relu is simpler. Network with the ReLU function tends to show better convergence (As shown in this specific task in the results section). However, ReLU can cause activations to grow very large when many inputs are positive (there are no restrictions on positive values), a problem that does not occur with sigmoid. ReLU also suffers from the “dying ReLU” problem, where many negative inputs produce zero gradients, preventing learning in the affected neurons.

Softmax

Softmax function: $s(x_i) = \frac{e^{x_i}}{\sum_{j=1}^n e^{x_j}}$

While testing the network with softmax as the output activation, numerical instability issues arose during loss computation. The exponential value of the Z value “ $\text{np.exp}(Z)$ ” becomes a very high or very small number which then when calculating the logarithm of that value in the loss function it produces a very high (overflow) or small number (underflow). The numerical issue was solved by subtracting the maximum value of each row from the input logits before exponentiation, exploiting the shift-invariance of softmax to prevent overflow and underflow while preserving correct probabilities.

2.2 Fully parametrizable neural network

The neural network class implementation uses activation functions (for both forward and backward passes) and loss functions defined in another Python script (`dnn_functions.py`). The class includes several tunable hyperparameters, data structures (such as dictionaries and arrays) to store and update important training information, and private functions that support public methods callable by the user. All the hyperparameters of the network have a

default value assigned, in case the user forgets to specify with the exception of the network architecture which must be declared and passed, it is explained in the following section.

2.2.1 Architecture of the network

An important hyperparameter is the one defining the architecture of the network. The parameter (`hidden_layers_units`) is an array of numbers, where the numbers represent the amount of nodes that each layer has and the amount of numbers that the array contains represents the amount of layers. For instance “[33, 16, 32, 5]” means that there are two hidden layers, the first contains 16 nodes, the second 32 nodes, the first layer implies that there are 33 features and the output layer predicts 5 classes. Note that the parameter represents the whole architecture of the network, input layer hidden layers and output layer included.

2.2.2 Basic hyperparameters

Furthermore, it is possible to tune the amount of epochs that the training will run for (**epoch**), the **learning rate** at which the network will learn (update weights and bias), it is possible to tune the **seed** to replicate the results (used during the weights initialisation and mini_batch random creation). Additionally it is possible to change the **hidden activation function** (either ReLU or Sigmoid) and **the output activation function** (either Sigmoid or Softmax since the task is classification) of the network. Besides, it is also possible to change the **loss function**, between **categorical cross entropy** (CCE) and **binary cross entropy** (BCE). Last but not least it is also possible to tune the verbose hyperparameter to see further logs during the training of the network (by default 0).

2.2.3 Regularisation

1. Dropout / Inverted Dropout

Dropout and inverted dropout are implemented during the forward pass, by generating a random mask of deactivated and activated neurons in every layer. We then store each mask in an array that is used during the backward pass to drop the same set of neurons used during the forward pass. In both passes, inverted dropout is taken into account by scaling up the non-deactivated neurons; this avoids us to scale during evaluation/testing of the model. The amount of nodes that are turned off during training (**dropout rate**) is also a hyperparameter tunable, by default there is no dropout. (rate of 0).

2. L1/L2 Regularisation

It is possible to apply L1 and/or L2 regularisation to the network class, through the hyperparameters **lambda_l1** and **lambda_l2** respectively. If the user decides to use any of those regularisers it is needed to set the lambda values to a value that ranges from 0 to 1, by default both lambdas are set to 0, so no regularisation is applied. This technique is applied to the calculation of the loss function and during the backward pass, to update the derivative of the weights.

2.2.4 Optimizers

There are two optimizers successfully implemented; momentum and mini batch gradient descent. Stochastic gradient descent (SGD) has also been implemented but unsuccessfully, it provokes a very slow training with our current dataset. Besides, the default optimizer is regular gradient descent, it is possible to combine gradient descent with mini-batch or momentum with mini-batch gradient descent.

1. Momentum

Momentum is a very useful optimizer that avoids the gradient to get stuck in a local optima by taking into account all the past gradients in the velocity vector. To set it in the network, it is mandatory to set the value “**momentum**” to the **optimizer1** parameter. Momentum uses a **beta1** value, which by default is set to 0.9, it does the average of the last 10 iterations of the gradient.

2. Mini Batch Gradient Descent

This optimizer is extremely useful, it can reduce bias, improve generalisation, there is faster convergence, it has an efficient memory usage and it can reduce noise. There is a **mini_batch** boolean parameter that, when set to True, uses mini-batches instead of processing the entire dataset (it is set to False by default). The size of the batches is controlled by the **mini_batch_size** parameter, which is set to 32 by default.

2.3 Results and Discussion

“After completing development, I conducted extensive hyperparameter tuning and testing, recording all experimental results in an Excel file (‘hyperparameters_tuning_results.xlsx’), which enabled systematic comparison across different model configurations. As metrics I used the accuracy for both testing and training, which helped assess whether the models were generalising correctly, underfitting or overfitting. Besides I have used the training processing time, since the results of our tuning has been surprisingly good I have tried to find the most efficient model trained with the least amount of epoch. I have tried different Architectures, different learning rates, tried switching optimizers and combining them, applying different kinds of regularisation techniques. For our discussion I will be using our final model, with the best set of hyperparameters, which you can see in the Final Results section.

Different Architectures

Based on the obtained results, the problem appears to be almost linearly separable, which was unexpected given that the original dataset contains 42 features, reduced to 33 after preprocessing. This observation was made too late in the process to switch to a different dataset for a third time. As evidence of the last observation, regardless of the network architecture, the achieved accuracy consistently ranges between 96% and 98% on both the training and test sets. Impressively, even a logistic regression model (without hidden layers) performs very well. Nevertheless, the inclusion of a single hidden layer leads to a small but consistent improvement over logistic regression (see table below). The following models exposed in the table have used the following hyperparameters: learning rate=0,05, hidden activation=relu, output activation= softmax, as optimizer regular gradient descent and mini batches with size 16 running on 100 epoch. In the table

Architecture	Train accuracy	Test accuracy
[33, 3] (Logistic regression)	0,98418	0,982
[33,16,3]	0.982875	98225
[33, 16, 8, 3]	0,9852	0,985
[33, 32, 16, 3] (Best)	0,98651	0,98525

Table 1: Different architectures Results

Optimizers

The best model uses regular gradient descent parameters update and mini-batch gradient descent, this leads us to extremely low training processing time and the highest train and test accuracies. Momentum optimizer reduces our accuracy in both sets by a very little percentage, since the oscillation is greater, it does not converge as cleanly as gradient descent. Mini batch not only trains the model way quicker than without, it also improves the accuracy of both sets, it adds a bit of noise but ultimately converges faster. We can see in the table below how the accuracy and test time varies between models (they use the best architecture explained above). We can also see how momentum increases the accuracy over gradient descent parameters updates and also reduces the training time since the gradient moves faster and more consistently generalising better. Note that the size of the mini batches is also an important hyperparameter, which we will not be discussing but the most optimal is size 16.

Optimizers	test accuracy	Training time
Mini batch gd + gradients regular update	0,98525	1,2 sec
No mini batches + gradients regular updates	0,8183	14,1 sec
Mini batch gd + momentum update	0.9832	0,92 sec
No mini batches + momentum update	0,88255	11,1 sec

Table 2: Different optimizers Results

Relu vs Sigmoid

The ReLu activation function achieves better results than using the Sigmoid function, faster convergence is a property of it. We can see the comparison on the following table, tested over 1000 epochs, the difference in accuracy is smaller using mini batches but it is considerable without mini batches. The same goes for the training time, the sigmoid function takes longer to compute as a result of the exponential operation.

Activation function and/no mini batches	Test accuracy	Training accuracy	Training Time
Relu + mini batches	0,986075	0,985	5,587888
Sigmoid + mini batches	0,9547	0,95215	9,000815
Relu without mini batches	0,958288	0,95805	141,7029
Relu without mini batches	0,783438	0,77975	261,0108

Table 3: Different optimizers Results

Regularisation techniques

The training accuracy and the test accuracy have been consistent across iterations which implies there is no overfitting nor underfitting which suggests that there is no need of adding dropout nor L1/L2 regularisation. As a matter of fact by adding any of them the accuracy drops as a result.

Final Results

Our final model uses as architecture 33 input nodes, 2 hidden layers (32 nodes and 16 nodes respectively) and an output layer of 3 nodes. Uses the ReLU activation function for the hidden layers and the Softmax function for the output layer (several classes), it also uses the categorical cross entropy loss. It does not use any regularisation techniques (no Dropout nor L1/L2). It is trained with a learning rate of 0.05 which may seem high but it converges good and is stable across tests. It is trained across 100 epochs and uses mini batch gradient descent as optimizer with regular gradient update.

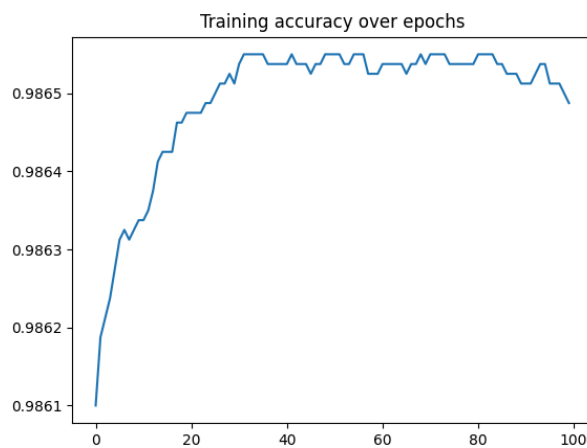


Figure 1: Training accuracy over epochs

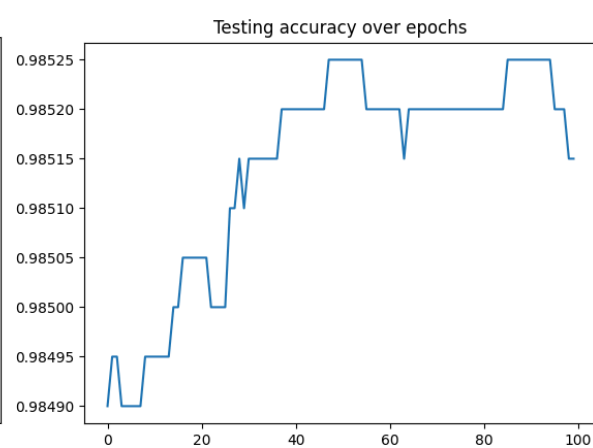


Figure 2: Test Accuracy over epochs

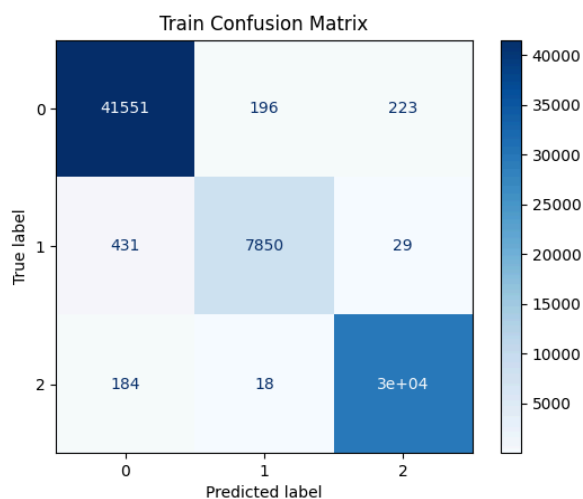


Figure 3: Train confusion Matrix

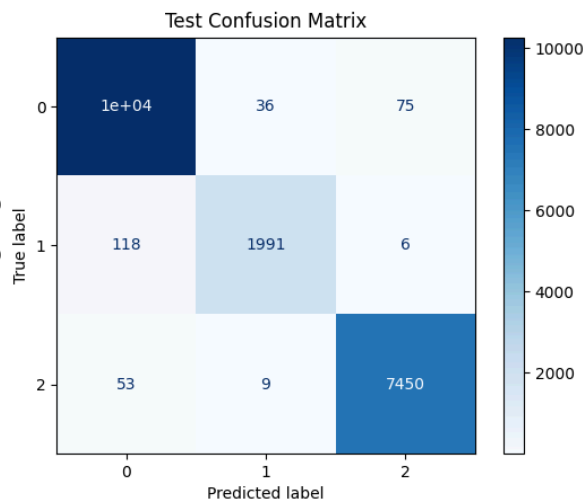


Figure 4: Test confusion Matrix

The final model reaches a final test accuracy of 0.98515, a train accuracy of 0.9864 and a loss of 0.00759 in a total training time of 5,573 seconds. The accuracy starts at 0.98 from the first epoch because the dataset is highly separable, the model has sufficient capacity, and

the combination of a high learning rate, softmax with categorical cross-entropy, and no regularization allows the network to converge very rapidly during the first full pass over the data. Besides we can see in the confusion matrixes that the model predicts very well all three classes even if there is an imbalance problem. I believe we have reached a ceiling and it is not possible to further improve our model. The dataset seems very clean, with little noise and the data almost linearly separable which explains the results reached. I believe it would have been better to test the network with a different dataset with more room for improvement.

3. Task 2 - Building Neural networks with PyTorch

First of all I have done some pre-processing to ensure that the tests run smoothly, using all resources and tools available to make the test of different networks efficient. For this I have used DataLoader which groups the samples of the dataset into batches, it shuffles the data and most importantly it loads it in parallel, feeding the data efficiently to the GPU which helps achieving a faster GPU computation and more stable gradients. I have also used my local Nvidia GPU to do the training of the different models.

3.1 First Model: Neural Network

For a first model approach I used fully connected neural networks, following the implementation carried out in the first task I wanted to experiment on how a neural network performs in image classification with a complex amount of images and categories. However, as expected, the model was extremely slow and did not perform very well.

3.2 Architecture Improvement: Convolutional Neural Network

Since the dataset is composed of images and taking into account the bad results of running a fully connected neural network on it I decided to implement a whole different model, a CNN, which is the way to go in this situation. A convolutional neural network is the technique used when working with images, for object detection, image segmentation, object recognition and in this case aerial landscapes image classification. The architecture of the model has varied across tests during the search for the best fit, however they all share a sequential set of convolutional layers following a ReLu activation function and MaxPoolLayers. The CNN model follows a very simple fully connected layer that is in charge to do the final classification of the landscapes.

3.3 Results and discussion

Regular Neural Network Results

Due to the initial neural network being computationally expensive and performing poorly at low epoch counts, most hyperparameter tuning focused on improving the Convolutional Neural Network model. Nevertheless I have run several iterations with the first model, tuning different network architectures and hyperparameters, different optimizers and different number of epochs and have found that there is a limitation with this model. A regular (fully connected) neural network applied to a big image dataset results in a very large number of parameters, which leads to inefficient and slow learning. Although the loss continues to decrease during training (Figure 5), the accuracy quickly reaches a plateau and fails to

improve further which means decision boundaries are not improving. This indicates that the model is increasing its confidence on already correctly classified samples, without learning additional discriminative features that would improve overall classification performance. The final accuracy reached on our best NN model is **0.3442**, with 50 epochs although the accuracy stops improving from the epoch 16.

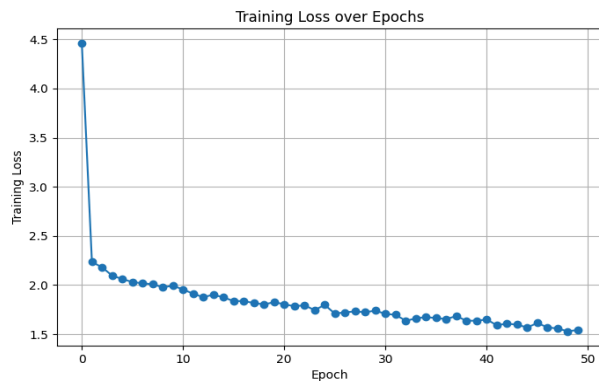


Figure 5: NN: loss over epochs

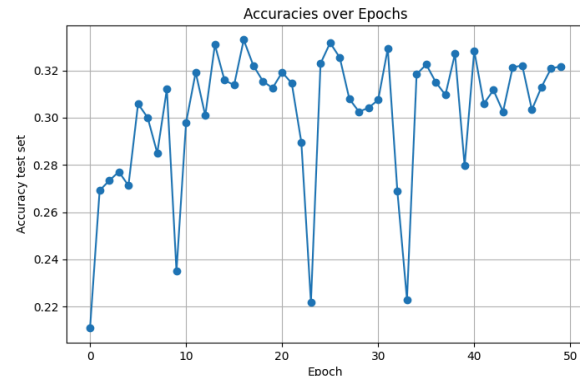


Figure 6: NN: accuracy over epochs

Convolutional Neural Networks Results

The first results that I obtained with a CNN model composed of two convolutional layers followed by a sequential fully connected network using the Adam optimizer with a single epoch was a test accuracy of **0.54%**, which already outperforms our previous best model. The accuracy kept improving as I added more layers, up to the fifth convolutional layer, after which increasing depth no longer significantly improved the model. A possible explanation is that more depth means that there are more representative features (edges, textures, parts), this last model obtained a total accuracy of **0.74%**. It is important to note that for every Convolutional network that I added to the sequence, I also added a ReLu activation function (to avoid linearity) and a MaxPool2d layer to downsample the feature space. The table below shows these results over 10 epochs.

Architectures CNN	Final loss	Test accuracy	Processing time (s)
2 Conv2d Layers	0.0482	0.6012	467.56
3 Conv2d Layers	0.0697	0.6704	419.09
4 Conv2d Layers	0.0846	0.7125	452.34
5 Conv2d Layers	0.2000	0.7433	817.50

Table 4: Different CNN architectures results

I have also tried several learning rate values but the decisive one is 0.001, by adding a dropout of 0.5 to the fully connected layer the accuracy improved further to a total of 0.77%. Since dropout adds a bit of noise to the training generally to prevent overfitting it can increase test accuracy by generalising better instead of focusing on strong features. Finally I have also added an extra layer called Adaptive Average Pooling layer at the end of the CNN sequence, to reduce parameters that are then fed to the MLP Flatten layer and at the same time it helps improve generalisation. By further trials I have determined that the fifth layer was not necessary, therefore our Final model architecture and hyperparameters would be the following:

The model was trained using the Adam optimizer for 50 epochs with a learning rate of 0.001 and categorical cross-entropy loss. The final CNN architecture consists of four convolutional blocks (Conv2d + ReLU + MaxPool2d), followed by an AdaptiveAvgPool2d layer and a classifier composed of Flatten, Linear, ReLU, Dropout of 0.5, and a final Linear layer.

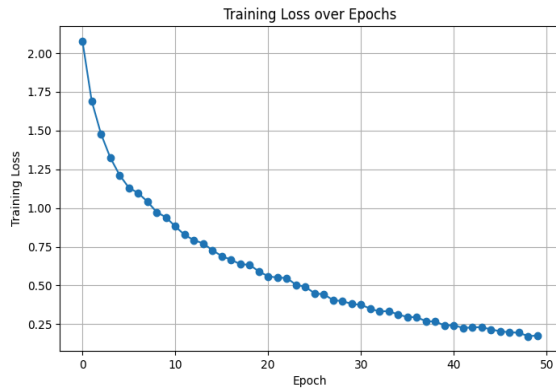


Figure 7: CNN: loss over epochs

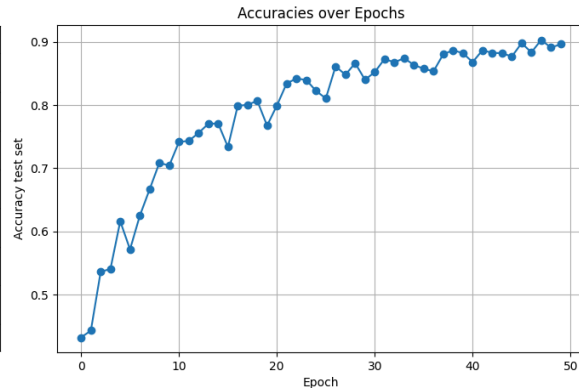


Figure 8: CNN: accuracy over epochs

Comparison of final results between the two models:

Final Models	Final Loss	Test Accuracy	Processing Time (s)
Neural Network	1.5421	0.3442	2548,23
CNN	0.17652	0.896	3040.25

Table 5: NN vs CNN Results

We can clearly see that there is a massive improvement from training a regular neural network to use a complex convolutional neural network to classify images. We can see in Figure 4 that the accuracy improves over time as the loss decreases (Figure 3), reaching a final test accuracy of **0.896** in contrast the NN model performed a maximum of 0.344. Even if the CNN model takes 8 more minutes than the regular NN model, we can say it is worth the wait. I believe there is still room for improvement and even reach at least a 0.98 accuracy with the CNN model, but rather than tuning hyperparameters and changing the architecture more transformation could be performed on the data in the pre-processing such as data-augmentation, Random horizontal flips or resize the data. A more complex optimizer might help such as AdamW and maybe adding early stopping and train with further epochs.

4. References

Most of my references during the development of the first task have been based on two assessments that I did in the past for an online course. I will put down the reference to the specific course. I have also used for both task references

References for Task 1

Coursera. (2019). *Deep Learning* / Coursera. [online] Available at: <https://www.coursera.org/specializations/deep-learning>.

Coursera. (2020). *Neural Networks and Deep Learning*. [online] Available at: <https://www.coursera.org/learn/neural-networks-deep-learning?specialization=deep-learning>.

Coursera. (n.d.). *Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization*. [online] Available at: <https://www.coursera.org/learn/deep-neural-network?specialization=deep-learning>.

DeepLearning.AI. (2022). *Feedforward Neural Networks in Depth*. [online] Available at: <https://community.deeplearning.ai/t/feedforward-neural-networks-in-depth/98811>.

Prajapati, J. (2025). *Softmax Pitfalls: Why Your Model Might Be Overconfident and How to Fix It*. [online] Medium. Available at: <https://medium.com/@jig4physics/softmax-pitfalls-why-your-model-might-be-overconfident-and-how-to-fix-it-7598bd14780f> [Accessed 27 Dec. 2025].

Denny's Blog. (2015). *Implementing a Neural Network from Scratch in Python*. [online] Available at: <https://dennybritz.com/posts/wildml/implementing-a-neural-network-from-scratch/>.

ihali95 (2019). *Building two layer neural networks from scratch*. [online] Kaggle.com. Available at: <https://www.kaggle.com/code/ihali95/building-two-layer-neural-networks-from-scratch>.

Cross Validated. (n.d.). *machine learning - What are the advantages of ReLU over sigmoid function in deep neural networks?* [online] Available at: <https://stats.stackexchange.com/questions/126238/what-are-the-advantages-of-relu-over-sigmoid-function-in-deep-neural-networks>.

Shukla, S.P. (2024). *Understanding Activation Functions: ReLU vs Sigmoid*. [online] Medium. Available at: <https://medium.com/%40shuklashubh818/understanding-activation-functions-relu-vs-sigmoid-0b24fd636ff9>.

References for Task 2

Intro (2022). *Quickstart — PyTorch Tutorials 2.8.0+cu128 documentation*. [online] Pytorch.org.

Available at: https://docs.pytorch.org/tutorials/beginner/basics/quickstart_tutorial.html

Pytorch.org. (2024). *Datasets & DataLoaders — PyTorch Tutorials 2.7.0+cu126 documentation*.

[online] Available at: https://docs.pytorch.org/tutorials/beginner/basics/data_tutorial.html.

Ugama Benedicta Kelechi (2025). *Understanding PyTorch's DataLoader: How to Efficiently Load and Augment Data*. [online] Medium. Available at:

<https://medium.com/@ugamakelechi501/understanding-pytorchs-dataloader-how-to-efficiently-load-and-augment-data-c9eb26f61491>

Pytorch.org. (2024). *ImageFolder — Torchvision main documentation*. [online] Available at:

<https://docs.pytorch.org/vision/main/generated/torchvision.datasets.ImageFolder.html>.

Pytorch.org. (2024). *torch.utils.data — PyTorch 2.7 documentation*. [online] Available at:

<https://docs.pytorch.org/docs/stable/data.html#torch.utils.data.DataLoader>.

Reference API (2025). *torch.utils.data — PyTorch 2.9 documentation*. [online] Pytorch.org. Available

at: https://docs.pytorch.org/docs/stable/data.html#torch.utils.data.random_split

Yadav, A. (2024). *PyTorch's Sequential - We Talk Data - Medium*. [online] Medium. Available at:

<https://medium.com/we-talk-data/pytorchs-sequential-3974f27c714e>.

Mayur Ingle (2024). *Simple Convolutional Neural Network (CNN) for Dummies in PyTorch: A*

Step-by-Step Guide. [online] Medium. Available at:

<https://medium.com/@myringoleMLGOD/simple-convolutional-neural-network-cnn-for-dummies-in-pytorch-a-step-by-step-guide-6f4109f6df80>.

Python API (2025). *AdaptiveAvgPool2d — PyTorch 2.8 documentation*. [online] Pytorch.org. Available

at: <https://docs.pytorch.org/docs/stable/generated/torch.nn.AdaptiveAvgPool2d.html>.

Stack Overflow. (n.d.). *What is Adaptive Average Pooling and how does it work?* [online] Available at:

<https://stackoverflow.com/questions/58692476/what-is-adaptive-average-pooling-and-how-does-it-work>.

Pytorch.org. (2024). *Training a Classifier — PyTorch Tutorials 2.7.0+cu126 documentation*. [online]

Available at: https://docs.pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html.

Cross Validated. (n.d.). *machine learning - What are the advantages of ReLU over sigmoid function in deep neural networks?* [online] Available at:

<https://stats.stackexchange.com/questions/126238/what-are-the-advantages-of-relu-over-sigmoid-function-in-deep-neural-networks>.

Basant C (2025). *Demystifying nn.AdaptiveAvgPool2d in PyTorch: Why Adaptive Pooling Matters in Deep Learning*. [online] Medium. Available at:
https://medium.com/@caring_smitten_gerbil_914/demystifying-nn-adaptiveavgpool2d-in-pytorch-why-adaptive-pooling-matters-in-deep-learning-1f7b7b1cc9b0.